

Project 02 — Advanced Docker CI/CD

Flask · Multi-stage Docker · PostgreSQL · Compose · GitHub Actions · Docker Hub

Build Walkthrough · saftab4-arch · 23 steps

Phase 1 - Scaffold the Project

- 01 Create the project directory
- 02 Create the app/ and tests/ folders
- 03 Create the source files
- 04 Review the full project structure

Phase 2 - Containerize (Multi-stage, Non-root)

- 05 Write a multi-stage Dockerfile
- 06 Run tests, then build the image
- 07 Run the container and test it
- 08 Verify the app in the browser
- 09 Confirm it runs as a non-root user

Phase 3 - Add PostgreSQL with Docker Compose

- 10 Author docker-compose.yml
- 11 Add the PostgreSQL healthcheck
- 12 Bring the stack up (and fix a port conflict)
- 13 Verify the custom network
- 14 Confirm the database is healthy
- 15 Verify the app via the Compose stack

Phase 4 - Author & Finalize the CI/CD Workflow

- 16 Draft the GitHub Actions workflow
- 17 Complete the workflow (first version)
- 18 Finalize triggers + fix the test job
- 19 Point the build job at the subfolder

Phase 5 - Pipeline Run & Publish

- 20 Pipeline passes end to end
- 21 Image published to Docker Hub

Phase 6 - Deploy Anywhere

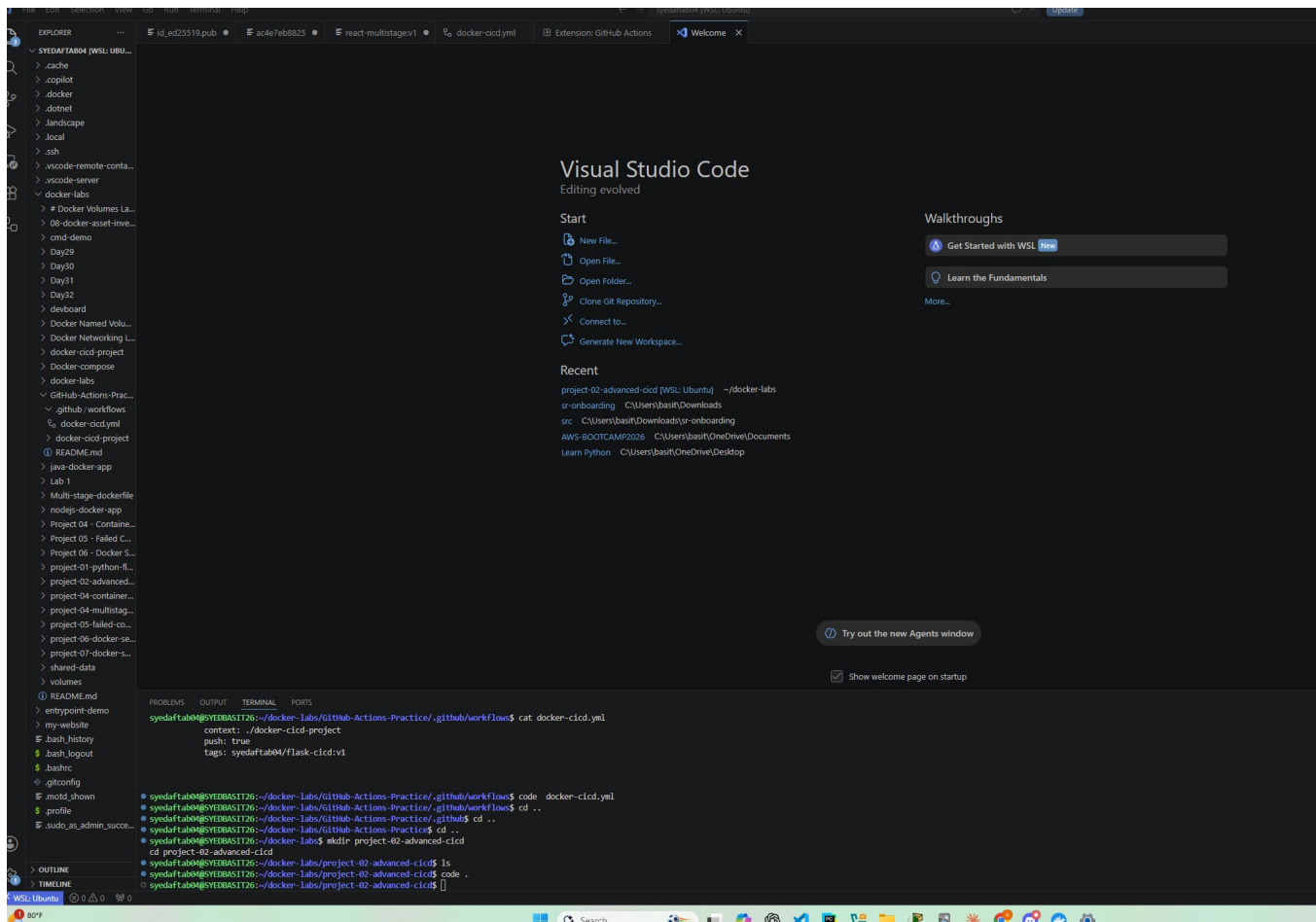
- 22 Pull and run the published image

23 Verify the deployment

PHASE 1 - SCAFFOLD THE PROJECT

Step 01 — Create the project directory

01-create-project-directory.jpg

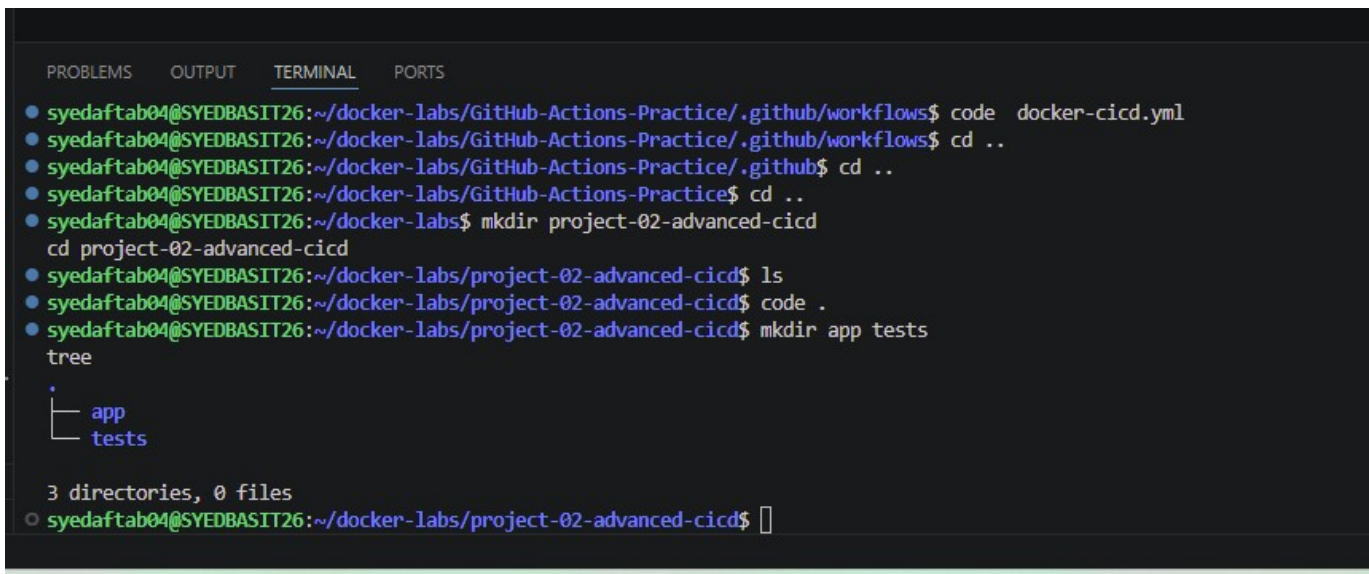


mkdir project-02-advanced-cicd, open it in VS Code with **code .**, and start the folder layout.

PHASE 1 - SCAFFOLD THE PROJECT

Step 02 — Create the app/ and tests/ folders

02-create-app-and-tests-folders.jpg



```
PROBLEMS OUTPUT TERMINAL PORTS
● syedaftab04@SYEDBASIT26:~/docker-labs/GitHub-Actions-Practice/.github/workflows$ code docker-cicd.yml
● syedaftab04@SYEDBASIT26:~/docker-labs/GitHub-Actions-Practice/.github/workflows$ cd ..
● syedaftab04@SYEDBASIT26:~/docker-labs/GitHub-Actions-Practice/.github$ cd ..
● syedaftab04@SYEDBASIT26:~/docker-labs/GitHub-Actions-Practice$ cd ..
● syedaftab04@SYEDBASIT26:~/docker-labs$ mkdir project-02-advanced-cicd
  cd project-02-advanced-cicd
● syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ ls
● syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ code .
● syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ mkdir app tests
tree
.
├── app
└── tests

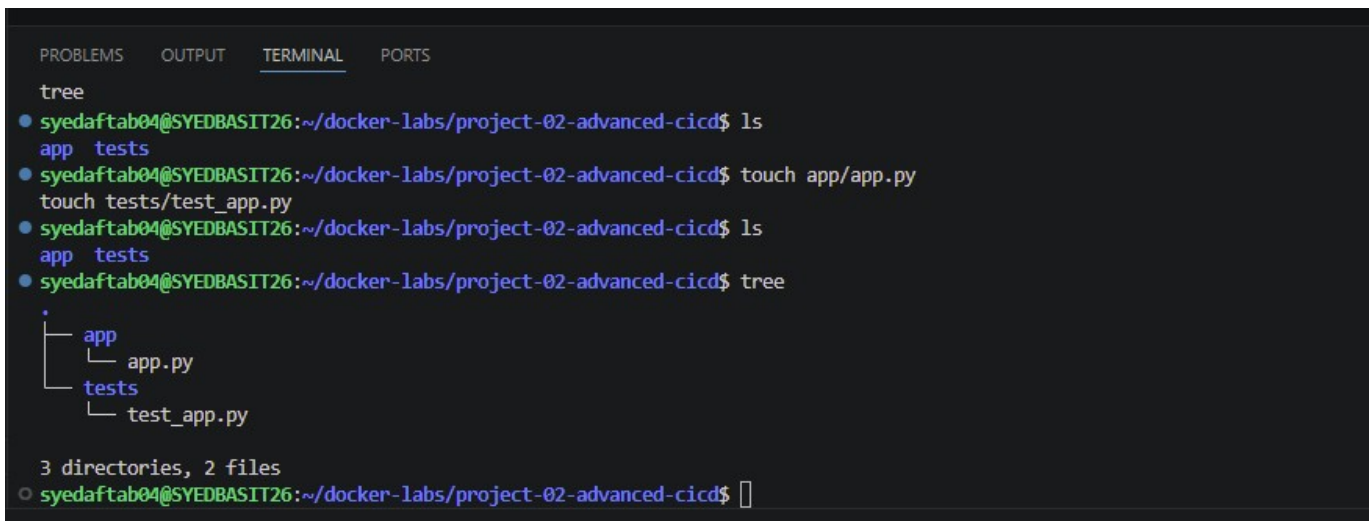
3 directories, 0 files
○ syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
```

mkdir app tests - separating application code from test code so pytest can target the package.

PHASE 1 - SCAFFOLD THE PROJECT

Step 03 — Create the source files

03-create-source-files.jpg

A terminal window with tabs for PROBLEMS, OUTPUT, TERMINAL, and PORTS. The TERMINAL tab is active. The user is in a directory ~/docker-labs/project-02-advanced-cicd. They run 'ls' showing 'app' and 'tests' directories. Then they run 'touch app/app.py' and 'touch tests/test_app.py'. Another 'ls' shows the files created. Finally, they run 'tree' which shows a directory tree with 'app' containing 'app.py' and 'tests' containing 'test_app.py'. The summary shows 3 directories and 2 files.

```
tree
syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ ls
app  tests
syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ touch app/app.py
touch tests/test_app.py
syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ ls
app  tests
syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ tree
.
├── app
│   └── app.py
└── tests
    └── test_app.py

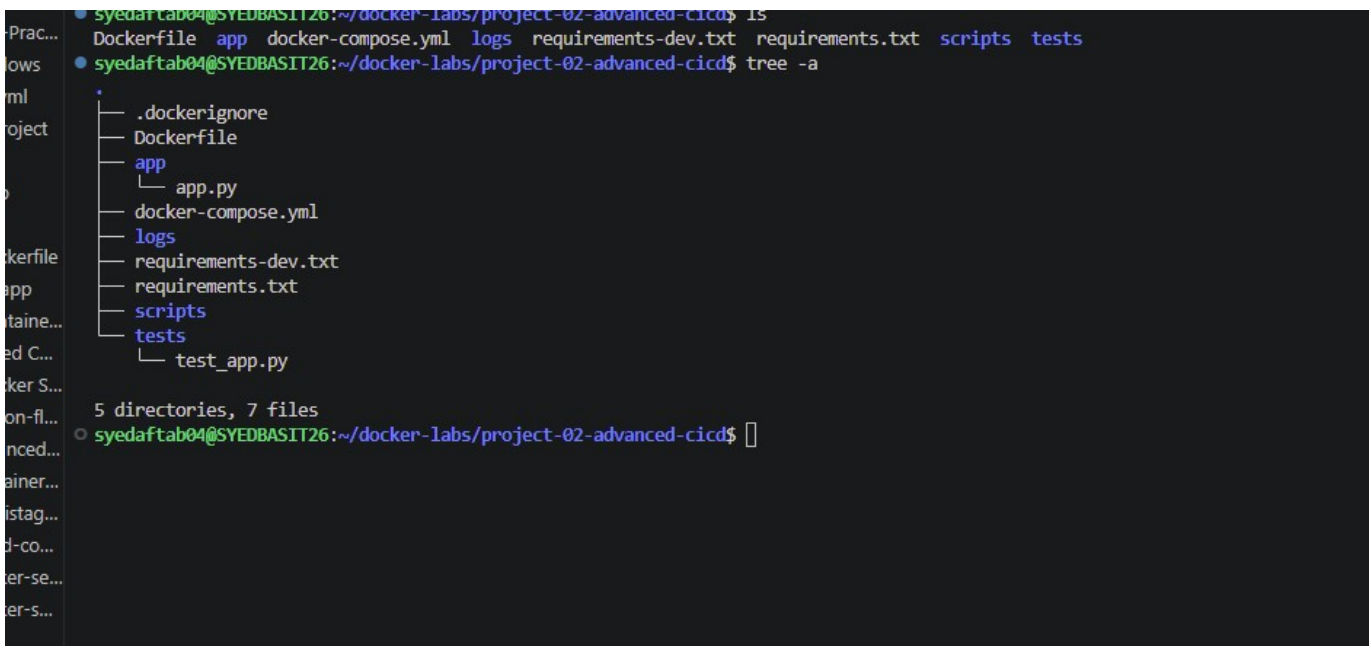
3 directories, 2 files
syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
```

touch app/app.py and **tests/test_app.py** - the Flask app plus its test module.

PHASE 1 - SCAFFOLD THE PROJECT

Step 04 — Review the full project structure

04-project-structure-tree.jpg

A terminal window showing the full project structure. The user runs 'ls' showing 'Dockerfile', 'app', 'docker-compose.yml', 'logs', 'requirements-dev.txt', 'requirements.txt', 'scripts', and 'tests'. Then they run 'tree -a' which shows a detailed directory tree including hidden files like '.dockerignore'. The summary shows 5 directories and 7 files.

```
syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ ls
Dockerfile  app  docker-compose.yml  logs  requirements-dev.txt  requirements.txt  scripts  tests
syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ tree -a
.
├── .dockerignore
├── Dockerfile
├── app
│   └── app.py
├── docker-compose.yml
├── logs
├── requirements-dev.txt
├── requirements.txt
├── scripts
├── tests
│   └── test_app.py
└──

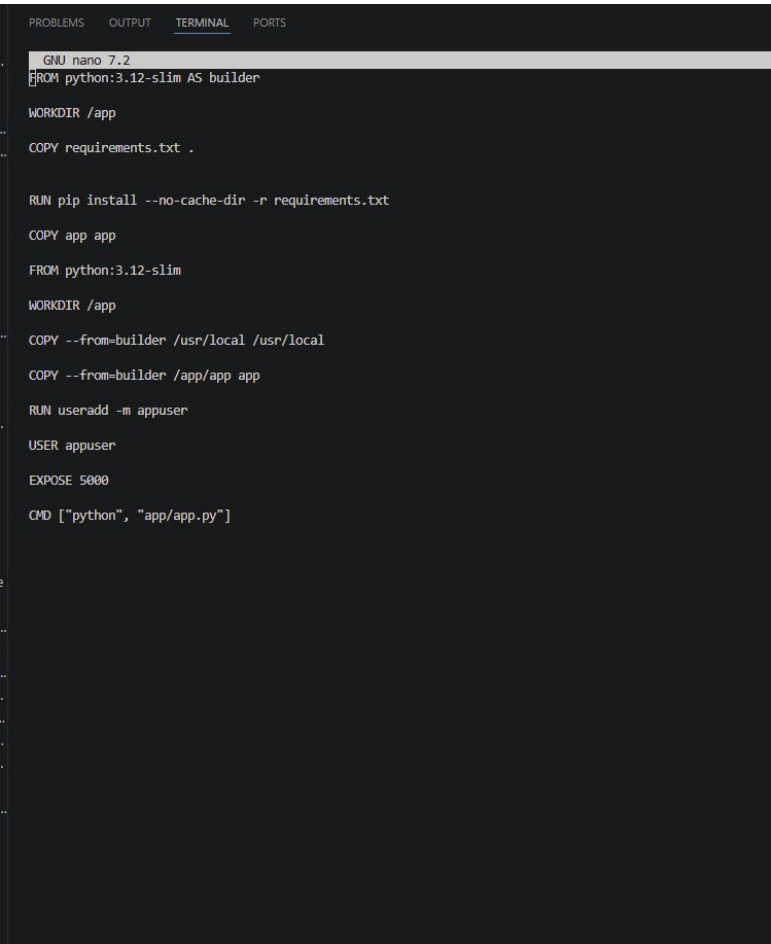
5 directories, 7 files
syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
```

tree -a shows the complete layout: Dockerfile, docker-compose.yml, app/, tests/, requirements.txt, requirements-dev.txt, and .dockerignore.

PHASE 2 - CONTAINERIZE (MULTI-STAGE, NON-ROOT)

Step 05 — Write a multi-stage Dockerfile

05-write-multistage-dockerfile.jpg



```
PROBLEMS  OUTPUT  TERMINAL  PORTS
GNU nano 7.2
FROM python:3.12-slim AS builder

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app app

FROM python:3.12-slim

WORKDIR /app

COPY --from=builder /usr/local /usr/local

COPY --from=builder /app/app app

RUN useradd -m appuser

USER appuser

EXPOSE 5000

CMD ["python", "app/app.py"]
```

A **builder** stage installs dependencies, the final stage copies only what is needed with **COPY --from=builder**, then **useradd appuser** + **USER appuser** drop root.

PHASE 2 - CONTAINERIZE (MULTI-STAGE, NON-ROOT)

Step 06 — Run tests, then build the image

06-run-tests-and-build-image.jpg

```
tests/test_app.py ..
===== 2 passed in 0.07s =====
• (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ nano Dockerfile
• (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ nano Dockerfile
• (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ nano Dockerfile
• (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ docker build -t advanced-cicd:v1 .
[+] Building 7.2s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 375B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 34B
=> [internal] load build context
=> => transferring context: 1.49kB
=> [builder 1/5] FROM docker.io/library/python:3.12-slim@sha256:d764629ce0ddd8c71fd371e9901efb324a95789d2315a47db7e4d27e78f1b0e9
=> resolve docker.io/library/python:3.12-slim@sha256:d764629ce0ddd8c71fd371e9901efb324a95789d2315a47db7e4d27e78f1b0e9
=> CACHED [builder 2/5] WORKDIR /app
=> [builder 3/5] COPY requirements.txt .
=> [builder 4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [builder 5/5] COPY app app
=> [stage-1 3/5] COPY --from-builder /usr/local /usr/local
=> [stage-1 4/5] COPY --from-builder /app/app app
=> [stage-1 5/5] RUN useradd -m appuser
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:e8a571414b33126b43ff54e7e4f406ccb4951385d60d7bc854364e62fc59f151
=> => exporting config sha256:d96c279a99ef3951485ba7d9096ed62ac668bce0b97e7acea11467d8b78ce993
=> => exporting attestation manifest sha256:59799d38947efb4e8b66be88a5988506d6382dfd51fffc67a6658d433f52e456
=> => exporting manifest list sha256:53ee15e3a20ac5bd468e2f0ab6b61e5281eddb4178d979d5712d7921ed545b7
=> => naming to docker.io/library/advanced-cicd:v1
=> => unpacking to docker.io/library/advanced-cicd:v1
• (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ docker images

IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
advanced-cicd:v1     53ee15e3a20a     214MB       52.6MB
alpine:latest        28bd5fe8b56d     13MB        3.93MB
cmd-demo:v1          f5bb6868889c     157MB       41.6MB      U
docker-demo:v1       0df0ad862e16     254MB       77.2MB      U
entry-demo:v1        620f65ee5e58     157MB       41.6MB      U
flask-cicd:v1         b111df087ca6     197MB       48.2MB      U
hello-world:latest   96498ff522e2     25.9kB      9.49kB
my-ubuntu:v1         ca915a9bd03      254MB       77.2MB      U
my-website:v1        8a4e965145ca     101MB       28.5MB      U
nginx:latest         42f2d24ae18d     241MB       66MB
postgres:latest      29ee7bb30d80     650MB       168MB
react-multistage:v1  e23246145e60     93.1MB      26.1MB
syedaftab04/react-multistage:v1 e23246145e60     93.1MB      26.1MB
ubuntu:latest        f3d28607ddd7     160MB       45.3MB

○ (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
```

pytest passes (2 passed), then **docker build -t advanced-cicd:v1 .** builds the image - confirmed in **docker images**.

PHASE 2 - CONTAINERIZE (MULTI-STAGE, NON-ROOT)

Step 07 — Run the container and test it

07-run-container-and-test.jpg

```
Run 'docker run --help' for more information
• (venv) syedafab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ docker rm 4428490ba32b
4428490ba32b
• (venv) syedafab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ docker run -d --name advanced-app-01 -p 5000:5000 advanced-cicd:v1
7c47cc8af17c0dd4587e1a30e4a8bb849433ba3f791ebf50514e7257c219e5ea
• (venv) syedafab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
7c47cc8af17c   advanced-cicd:v1  "python app/app.py"      4 seconds ago  Up 3 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  advanced-app-01
4f4142439804   my-website:v1    "/docker-entrypoint..." 21 hours ago   Up 21 hours   0.0.0.0:8010->80/tcp, [::]:8010->80/tcp      website21
3ca649ab3d3e   d7a627ec07fe    "/docker-entrypoint..." 21 hours ago   Up 21 hours   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp      website
• (venv) syedafab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ curl localhost:5000
Project 02 Advanced CI/CD(venv) syedafab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
• (venv) syedafab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
```

docker run -d --name advanced-app-01 -p 5000:5000 advanced-cicd:v1, verify with **docker ps**, and **curl localhost:5000** returns the app response.

PHASE 2 - CONTAINERIZE (MULTI-STAGE, NON-ROOT)

Step 08 — Verify the app in the browser

08-verify-app-in-browser.jpg

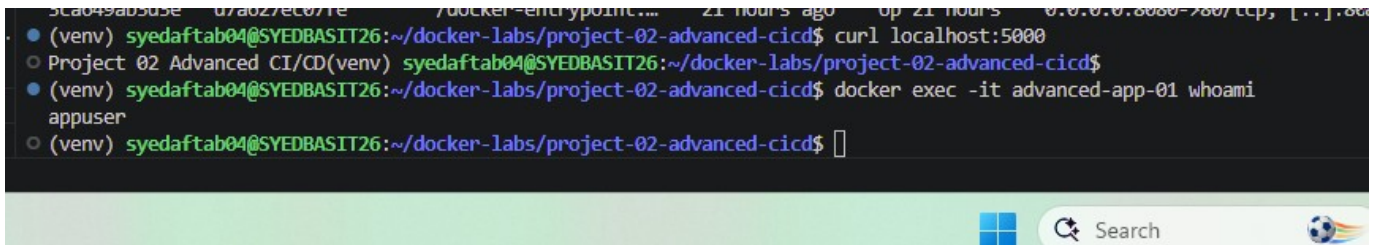


localhost:5000 renders **Project 02 Advanced CI/CD**.

PHASE 2 - CONTAINERIZE (MULTI-STAGE, NON-ROOT)

Step 09 — Confirm it runs as a non-root user

09-verify-running-as-non-root.jpg



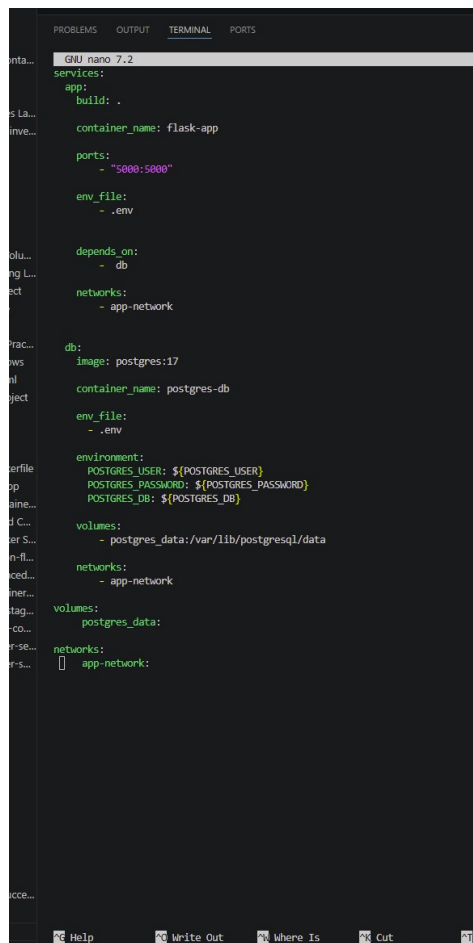
```
(venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ curl localhost:5000
Project 02 Advanced CI/CD(venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
(venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ docker exec -it advanced-app-01 whoami
appuser
(venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
```

docker exec -it advanced-app-01 whoami returns **appuser** - the hardening from the Dockerfile is working. Containers should never run as root.

PHASE 3 - ADD POSTGRESQL WITH DOCKER COMPOSE

Step 10 — Author docker-compose.yml

10-write-docker-compose.jpg



```
GNU nano 7.2
services:
  app:
    build: .
    container_name: flask-app
    ports:
      - "5000:5000"
    env_file:
      - .env
    depends_on:
      - db
    networks:
      - app-network

  db:
    image: postgres:17
    container_name: postgres-db
    env_file:
      - .env
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - app-network

volumes:
  postgres_data:

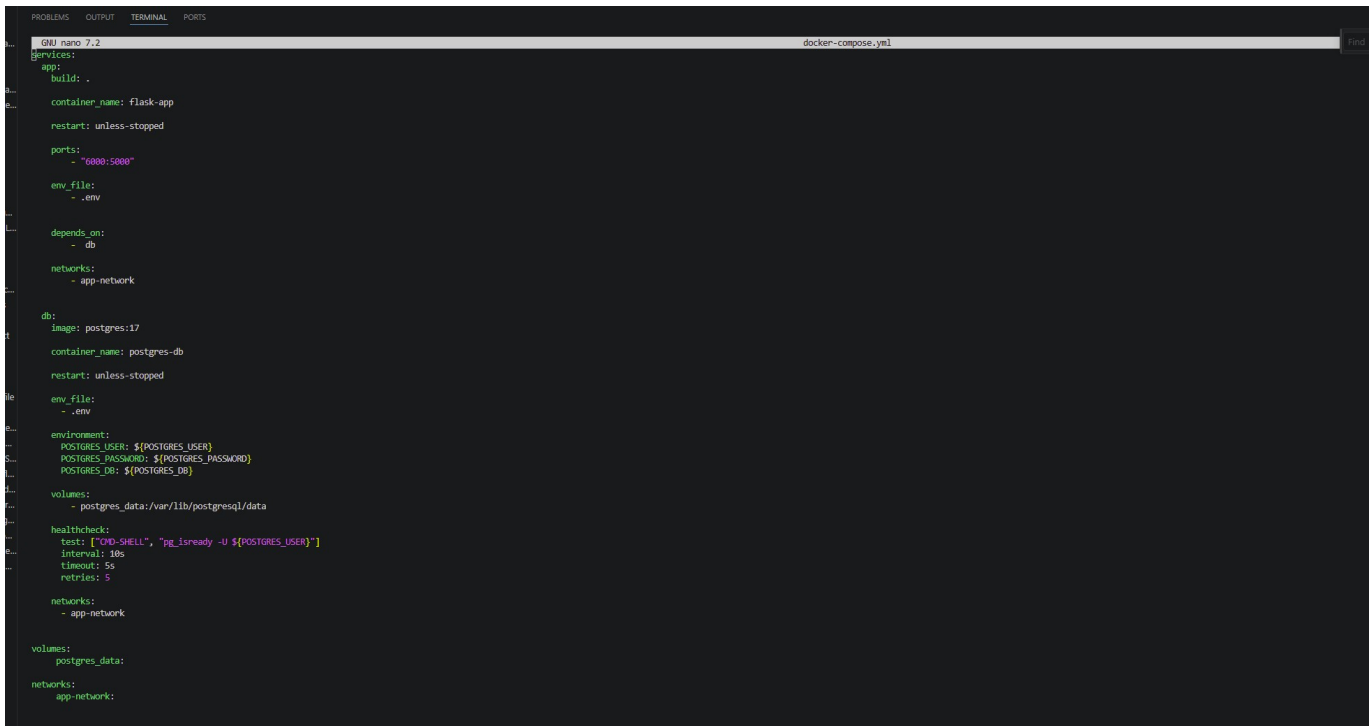
networks:
  app-network:
```

Two services - **app** (Flask) and **db** (postgres:17) - wired together on a shared **app-network**, with env files and a named volume for the database.

PHASE 3 - ADD POSTGRESQL WITH DOCKER COMPOSE

Step 11 — Add the PostgreSQL healthcheck

11-add-db-healthcheck.jpg

A screenshot of a terminal window with a dark background. The terminal shows the contents of a Docker Compose file named 'docker-compose.yml'. The file defines two services: 'app' and 'db'. The 'app' service is based on 'flask-app' and depends on the 'db' service. The 'db' service is based on 'postgres:17' and includes a healthcheck configuration. The healthcheck uses the 'pg_isready' command to verify the database is ready to accept connections. The terminal window has tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, showing the file content. The file content is as follows:

```
services:
  app:
    build: .
    container_name: flask-app
    restart: unless-stopped
    ports:
      - "6000:5000"
    env_file:
      - .env
    depends_on:
      - db
    networks:
      - app-network

  db:
    image: postgres:17
    container_name: postgres-db
    restart: unless-stopped
    env_file:
      - .env
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 40s
      timeout: 5s
      retries: 5
    networks:
      - app-network

volumes:
  postgres_data:

networks:
  app-network:
```

pg_isready healthcheck (interval/timeout/retries), **postgres_data** named volume, and **restart: unless-stopped** so the stack self-heals.

PHASE 3 - ADD POSTGRESQL WITH DOCKER COMPOSE

Step 12 — Bring the stack up (and fix a port conflict)

12-compose-up-and-fix-port-conflict.jpg

```
✓ Image postgres:17 Pulled
[*] Building 1.1s (15/15) FINISHED
-> [internal] load local bake definitions
-> => reading from stdin 582s
-> [internal] load build definition from Dockerfile
-> => transferring dockerfile: 375B
-> [internal] load metadata for docker.io/library/python:3.12-slim
-> [internal] load dockerignore
-> => transferring context: 3kB
-> [internal] load build context
-> => transferring context: 272B
-> [builder 1/5] FROM docker.io/library/python:3.12-slim@sha256:d746629ce8d48c71f4371e9981efb324a95789d2315a47db7e4d27e78f1b0e9
-> => resolve docker.io/library/python:3.12-slim@sha256:d746629ce8d48c71f4371e9981efb324a95789d2315a47db7e4d27e78f1b0e9
-> CACHED [builder 2/5] WORKDIR /app
-> CACHED [builder 3/5] COPY requirements.txt .
-> CACHED [builder 4/5] RUN pip install --no-cache-dir -r requirements.txt
-> CACHED [builder 5/5] COPY app .
-> CACHED [stage-1 3/5] COPY --from-builder /usr/local /usr/local
-> CACHED [stage-1 4/5] COPY --from-builder /app/app .
-> CACHED [stage-1 5/5] RUN useradd -m appuser
-> exporting to image
-> => exporting layers
-> => exporting manifest sha256:2f6e0735027a7693b7f5270a37f4d9540bcbda9dc93f914597f74cac012e09
-> => exporting config sha256:380234acc11a40b29f77656fcb72a02018163f2ab8856541df10299303ed
-> => exporting attestation manifest sha256:c6a75739e4b549ce7877986e38d3724b66d7f3462de523d280e3280381fa1f85
-> => exporting manifest list sha256:4404f750d34e5ae546e3bae07b631bcffdb809ea8665c3e575fcf890d1ff1fee
-> => naming to docker.io/library/project-02-advanced-cicd-app:latest
[*] up 20/21s to docker.io/library/project-02-advanced-cicd-app:latest
✓ Image postgres:17 Pulled
✓ Image project-02-advanced-cicd-app Built
✓ Network project-02-advanced-cicd-app-network Created
✓ Volume project-02-advanced-cicd-postgres-data Created
✓ Container postgres-db Started
✓ Container flask-app Starting
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint flask-app (b34f1f4cf42a75c51994f8d89b4b3c672c27251336ceacfcd81fab19619a89): Bind for 0.0.0.0:5000 failed: port is already allocated
• (venv) syedatlab@YEEDBASIT76:~/docker-labs/project-02-advanced-cicd$ docker-compose.yml
• (venv) syedatlab@YEEDBASIT76:~/docker-labs/project-02-advanced-cicd$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
87e518ff0cfc  postgres:17    "docker-entrypoint.sh"  37 seconds ago Up 35 seconds  5432/tcp                               postgres-db
7c42cda1f1c  advanced-cicd-v1 "python app/app.py"     About an hour ago Up About an hour  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  advanced-app-01
4f4142439884  my-website:v1  "/docker-entrypoint..." 22 hours ago  Up 22 hours   0.0.0.0:8010->8010/tcp, [::]:8010->8010/tcp  website21
3ca649ab3d3e  d7a627ec87fe  "/docker-entrypoint..." 22 hours ago  Up 22 hours   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  website
• (venv) syedatlab@YEEDBASIT76:~/docker-labs/project-02-advanced-cicd$ docker stop 87e518ff0cfc
87e518ff0cfc
• (venv) syedatlab@YEEDBASIT76:~/docker-labs/project-02-advanced-cicd$ docker rm 87e518ff0cfc
87e518ff0cfc
• (venv) syedatlab@YEEDBASIT76:~/docker-labs/project-02-advanced-cicd$ docker compose up -d
[*] up 2/2
✓ Container postgres-db Started
✓ Container flask-app Started
• (venv) syedatlab@YEEDBASIT76:~/docker-labs/project-02-advanced-cicd$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
ac13159af087  project-02-advanced-cicd-app        "python app/app.py"     7 seconds ago Up 6 seconds   5000/tcp, 0.0.0.0:6000->6000/tcp, [::]:6000->6000/tcp  flask-app
eab8517740257  postgres:17    "docker-entrypoint.sh"  7 seconds ago Up 6 seconds   5432/tcp                               postgres-db
7c42cda1f1c  advanced-cicd-v1 "python app/app.py"     About an hour ago Up About an hour  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  advanced-app-01
4f4142439884  my-website:v1  "/docker-entrypoint..." 22 hours ago  Up 22 hours   0.0.0.0:8010->8010/tcp, [::]:8010->8010/tcp  website21
3ca649ab3d3e  d7a627ec87fe  "/docker-entrypoint..." 22 hours ago  Up 22 hours   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  website
• (venv) syedatlab@YEEDBASIT76:~/docker-labs/project-02-advanced-cicd$
```

First compose up fails: **Bind for 0.0.0.0:5000 failed: port is already allocated**. Stop and remove the conflicting container, then **docker compose up -d** brings both up cleanly.

PHASE 3 - ADD POSTGRESQL WITH DOCKER COMPOSE

Step 13 — Verify the custom network

13-verify-docker-network.jpg

```
Usage: docker network

Run 'docker network --help' for more information
● (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
d7d884b56ee4        bridge              bridge              local
4c4560fc0c87        devops-net          bridge              local
d843fe92e413        host                host                local
dc6a8910d071        mynetwork           bridge              local
d85eb93ae557        none                null                local
6f35aadb0dca        project-02-advanced-cicd_app-network bridge              local
○ (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
```

docker network ls shows **project-02-advanced-cicd_app-network** - Compose created the isolated bridge the two services share.

PHASE 3 - ADD POSTGRESQL WITH DOCKER COMPOSE

Step 14 — Confirm the database is healthy

14-verify-db-healthy.jpg

```
pyenv.cfg
233 directories, 2349 files
● (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$ docker compose ps
NAME                IMAGE                COMMAND              SERVICE  CREATED          STATUS              PORTS
flask-app           project-02-advanced-cicd-app "python app/app.py"  app      41 seconds ago   Up 41 seconds      0.0.0.0:6000->5000/tcp, [::]:6000->5000/tcp
postgres-db         postgres:17          "docker-entrypoint.s..." db        41 seconds ago   Up 41 seconds (healthy)  5432/tcp
○ (venv) syedaftab04@SYEDBASIT26:~/docker-labs/project-02-advanced-cicd$
```

docker compose ps shows **postgres-db** as **(healthy)** - the healthcheck is passing.

PHASE 3 - ADD POSTGRESQL WITH DOCKER COMPOSE

Step 15 — Verify the app via the Compose stack

15-verify-app-via-compose.jpg

A screenshot of a web browser's address bar. It features navigation icons (back, forward, refresh) and an information icon (i) on the left. The address 'localhost:5000' is entered in the text field on the right.

Project 02 Advanced CI/CD

localhost:5000 served through the full Compose stack returns the app.

PHASE 4 - AUTHOR & FINALIZE THE CI/CD WORKFLOW

Step 16 — Draft the GitHub Actions workflow

16-author-workflow-draft.jpg

```
GNU nano 7.2
name: Project 02 CI/CD

on:
  push:
    paths:
      - "app/**"
      - "tests/**"
      - "Dockerfile"
      - ".github/workflows/cicd.yml"

jobs:
  lint:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install ruff

      - name: Run Ruff
        run: ruff check .

  test:
    needs: lint
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt

      - name: Run tests
        run: python -m pytest

  build:
    needs: test
```

Three jobs sketched - **lint** (ruff) then **test** (needs: lint) then **build** (needs: test) - with the build job still to be filled in.

PHASE 4 - AUTHOR & FINALIZE THE CI/CD WORKFLOW

Step 17 — Complete the workflow (first version)

17-complete-workflow-first-version.jpg

```
GNU nano 7.2

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Setup Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.12"

  - name: Install dependencies
    run: |
      pip install -r requirements.txt
      pip install ruff

  - name: Run Ruff
    run: ruff check .

test:
  needs: lint
  runs-on: ubuntu-latest

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Setup Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.12"

  - name: Install dependencies
    run: |
      pip install -r requirements.txt

  - name: Run tests
    run: python -m pytest

build:
  needs: test
  runs-on: ubuntu-latest

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Set up Docker Buildx
    uses: docker/setup-buildx-action@v3

  - name: Login to Docker Hub
    uses: docker/login-action@v3
    with:
      username: ${{ secrets.DOCKERHUB_USERNAME }}
      password: ${{ secrets.DOCKERHUB_TOKEN }}

  - name: Build and Push Docker image
    uses: docker/build-push-action@v6
    with:
      context: .
      push: true
      tags: syedaftab04/project-02:v1

[]
[?] Help      [?] Write Out  [?] Where Is   [?] Cut        [?] Execute   [?] L
[?] Exit      [?] Read File  [?] Replace    [?] Paste      [?] Justify   [?] G
```

Full build job added: Buildx setup, Docker Hub login via **secrets**, and **build-push-action** tagging **syedaftab04/project-02:v1** (context at repo root).

PHASE 4 - AUTHOR & FINALIZE THE CI/CD WORKFLOW

Step 18 — Finalize triggers + fix the test job

18-finalize-paths-and-working-directory.jpg

```
GAJ nano 7.2
name: Project 02 CI/CD

on:
  push:
    paths:
      - "project-02-advanced-cicd/app/**"
      - "project-02-advanced-cicd/tests/**"
      - "project-02-advanced-cicd/dockerfile"
      - ".github/workflows/cicd.yml"

jobs:
  lint:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          pip install -r project-02-advanced-cicd/requirements.txt
          pip install ruff

      - name: Run Ruff
        run: ruff check .

  test:
    needs: lint
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          pip install -r project-02-advanced-cicd/requirements.txt
          pip install -r project-02-advanced-cicd/requirements-dev.txt

      - name: Run tests
        working-directory: ./project-02-advanced-cicd
        run: python -m pytest

  build:
    needs: test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Docker buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }
```

Scope the **paths** to **project-02-advanced-cicd/**, install from **project-02-advanced-cicd/requirements.txt**, and set **working-directory** so pytest resolves the **app** module (the fix behind run #5).

PHASE 4 - AUTHOR & FINALIZE THE CI/CD WORKFLOW

Step 19 — Point the build job at the subfolder

19-finalize-build-job-subfolder.jpg

```

pip install -r project-02-advanced-cicd/requirements.txt
pip install -r project-02-advanced-cicd/requirements-dev.txt
- name: Run tests
  working-directory: ./project-02-advanced-cicd
  run: python -m pytest

build:
  needs: test
  runs-on: ubuntu-latest

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}

    - name: Build and Push Docker image
      uses: docker/build-push-action@v6
      with:
        context: ./project-02-advanced-cicd
        file: ./project-02-advanced-cicd/Dockerfile
        push: true
        tags: syedaftab04/project-02:v1

```

Build job updated: **context:** ./project-02-advanced-cicd, **file:** ./project-02-advanced-cicd/Dockerfile, **push:** true, tag **syedaftab04/project-02:v1**.

PHASE 5 - PIPELINE RUN & PUBLISH

Step 20 — Pipeline passes end to end

20-pipeline-success.jpg

saftab4-arch / GitHub-Actions-Practice

[Code](#) [Issues](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

← Project 02 CI/CD

✔ Fix pytest working directory #5

Summary

All jobs

lint

test

build

Run details

Usage

Workflow file

Triggered via push 1 minute ago

Status

Total duration

Artifacts

saftab4-arch pushed · b58cd6a main

Success

49s

1

cicd.yml

on: push

lint8s

test10s

build22s

build summary

Docker Build summary

For a detailed look at the build, download the following build record archive and import it into Docker Desktop's Builds view. Build records include details such as timing, dependencies, results, logs, traces, and other information about a build. [Learn more](#)

saftab4-arch-GitHub-Actions-Practice-UF7Y1.dockerbuild (42.38 KB - includes 1 build record)

Find this useful? [Let us know](#)

ID	Name	Status	Cached	Duration
UF7Y1	project-02-advanced-cicd	completed	0%	10s

► Build inputs

Job summary generated at run-time

Annotations

3 warnings:

lint

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: [https://github.blog/changelog/2025-](#)

test

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: [https://github.blog/changelog/2025-](#)

build

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, docker/build-push-action@v6, docker/login-action@v3, docker/setup-buildx-action@v3. Fo

70°F

Mostly clear

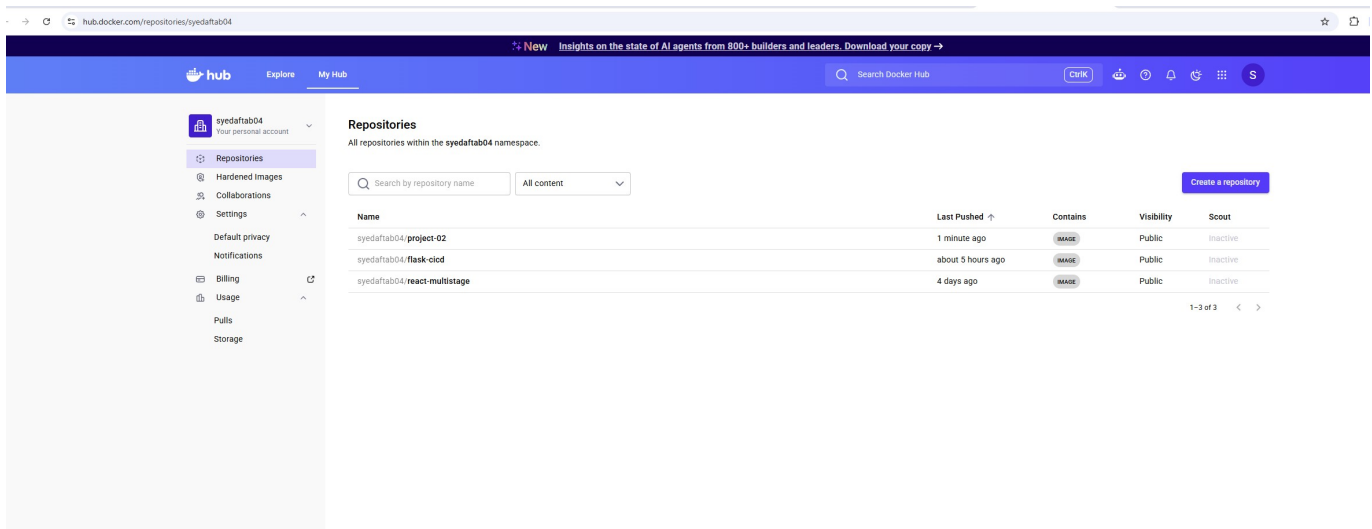
Search

Run #5 (Fix pytest working directory) is Success: lint -> test -> build all green. A single push now runs the whole chain automatically.

PHASE 5 - PIPELINE RUN & PUBLISH

Step 21 — Image published to Docker Hub

21-image-published-to-dockerhub.jpg

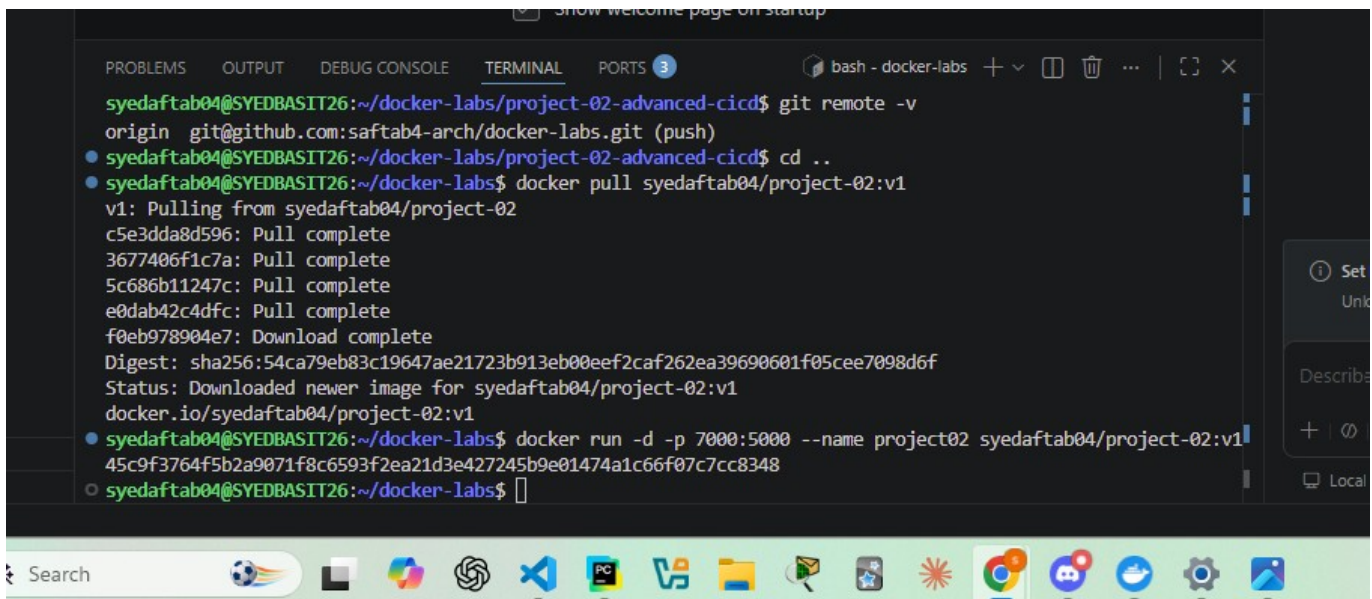


syedaftab04/project-02 appears in Docker Hub, pushed 1 minute ago straight from the GitHub Actions pipeline - no manual build or push.

PHASE 6 - DEPLOY ANYWHERE

Step 22 — Pull and run the published image

22-deploy-pull-and-run.jpg



docker pull syedaftab04/project-02:v1, then **docker run -d -p 7000:5000 --name project02** - deploying the exact image the pipeline built, on a fresh port.

PHASE 6 - DEPLOY ANYWHERE

Step 23 — Verify the deployment

23-verify-deployment.jpg

A screenshot of a web browser's address bar. It features navigation icons on the left: a back arrow, a forward arrow, and a refresh/circular arrow icon. To the right of these icons is a circular information icon (an 'i' inside a circle) followed by the text 'localhost:7000'. The entire address bar has a light blue background.

Project 02 Advanced CI/CD

localhost:7000 serves **Project 02 Advanced CI/CD** - code push to live container, fully automated. Project complete.